

On the Dangers of RSA Exponent Transforms

Eugene Lau, Laura Shea, and Nadia Heninger

Department of Computer Science and Engineering
University of California San Diego, USA
eylau@ucsd.edu, lmshea@ucsd.edu, nadiah@cs.ucsd.edu

April 22, 2026

Abstract. We analyze the security of RSA keys where the public exponent e is larger than $\varphi(N)$. While nearly all real-world applications of RSA use a small set of pre-determined constant values for e , the literature contains a number of constructions involving large special-form exponents. Examples include proposed countermeasures against Wiener’s attack on small RSA private exponents, exponent masking against side channels, a 2018 proposal by Joye and Michalevsky to extend the usefulness of hardware security modules, and a 2023 RSA blind signature construction by Amjad, Yeo, and Yung.

We give an efficient algorithm to factor an RSA modulus N given an integer a that is “close” to a multiple of $\varphi(N)$. That is, we can factor N in polynomial time given $\varphi(N) < a \leq N^{3/2}$ if there is an integer y with $|y| \leq aN^{-3/4}$ such that $a - y \equiv 0 \pmod{\varphi(N)}$. Our attack is a special case of Blömer and May’s 2004 algorithm using Coppersmith’s method that enables us to give stronger bounds for our application range of interest. We instantiate our attack against several constructions and exhibit families of weak public exponents that do not appear to have been analyzed in the literature. In particular, the Joye and Michalevsky exponent transform permits full key recovery if used for small public exponents. While it is well known that RSA is vulnerable for small private exponent d , our work suggests that care must also be taken when generating large public exponents, or when publishing transformed exponents.

1 Introduction

RSA is the most widely used digital signature scheme in practice for many use cases. While NIST has recently announced plans to deprecate RSA along with other quantum-vulnerable public-key cryptographic algorithms by 2035 [26], it is expected that many deployments will continue to use hybrid classical/post-quantum constructions for many years in the future.

RSA has been subjected to decades of cryptanalysis, and RSA public keys are typically generated following some common conventions. In practice, implementations almost always use small constant values for the public exponent e ; $e = 65537$ is overwhelmingly the most popular choice in practice. In fact, the US federal government’s FIPS 186-5 digital signature guidelines [14] require that

$2^{16} < e < 2^{256}$. Small values of e allow for more efficient signature verification, but implementers are nervous about *very* small RSA public exponents such as $e = 3$ because of Coppersmith’s attacks against small RSA public exponents with fixed-pattern encryption padding [11]; $e = 65537$ seems to strike a balance somewhere in the middle of these concerns.

For the private exponent d , a classic 1989 attack due to Wiener [35] shows how to factor an RSA modulus N using continued fractions given a public exponent e whose corresponding private exponent $d < N^{1/4}$. Boneh and Durfee improved this attack in 1999 [6] to $d < N^{0.292}$ by applying a bivariate version of Coppersmith’s method for finding small roots of polynomials modulo integers using lattice basis reduction. Requiring implementations to use a small public exponent e prevents the use of such small exponents d .

In this paper, we highlight a family of weak choices for the *public exponent* e when e is large. While not common in practice, there are a number of proposals in the academic literature that use customized or special-form values of e for special applications, including cases where $e > \varphi(N)$ or $e > N$.

We give a few examples:

- Wiener [35] suggests as a countermeasure to his small d attack to generate $e > N^{3/2}$ by adding multiples of $\varphi(N)$.
- Joye and Michalevsky [18] give an RSA exponent transform that allows users to convert RSA signatures generated using one exponent to a different exponent. The motivating application is to use a Hardware Security Module (HSM) that restricts the allowed class of public exponents to sign with an arbitrary choice of public exponent. Their construction maintains a secret HSM-only signature verification exponent e_s , and publishes the arbitrary public exponent e_p and a randomly chosen multiplier t that are related via $e_p t \equiv e_s \pmod{\varphi(N)}$. Here, $e_p t > \varphi(N)$.
- Amjad, Yeo, and Yung give several constructions for RSA blind signature schemes, some of which have motivated a recent IRTF draft [2]. One of their constructions [1, Appendix C] has potential exponent values of $\varphi(N) < e < N^{3/2}$.
- Lysanskaya, Miceli, Reyzin, and Shacham [21, Section 5] give a construction for sequential aggregate RSA signatures that uses $e > N$.
- Kocher [19] suggests masking exponents by adding a multiple of $\varphi(N)$ as a potential countermeasure against modular exponentiation-based side-channel attacks. This applies to the private exponent d but our work highlights that it *must only* apply to the private exponent.

1.1 Contributions

In this paper, we show that several of the above constructions may allow efficient private key recovery for a range of parameters. Our attack is independent of the size of d . In particular, we give the following (simplified) theorem.

Theorem 1 (Simplified). *Let $N = pq$ where p, q are primes of the same size. Given an integer a such that $\varphi(N) < a \leq N^{3/2}$ and $a - y \equiv 0 \pmod{\varphi(N)}$ for secret $|y| \leq aN^{-3/4}$, then N can be factored in polynomial time.*

Our theorem is a specialization and strengthening of a 2004 theorem of Blömer and May [5], and uses Coppersmith’s method for factoring from partial information via lattice basis reduction [11]. Interestingly, despite the numerous variants of Coppersmith-type theorems analyzing RSA constructions, parameter families, and partial key recovery problems [11, 10, 15, 7, 4, 13, 3, 32, 20, 9, 24, 36], we have not seen this particular version articulated as such.

In Section 7 we experimentally validate the range of a and y for which the above theorem holds. The experiments match, and slightly outperform, our proven theoretical results. The above simplified theorem is more explicitly stated as Theorem 5 of Section 3.

As a corollary to the above theorem, we obtain the following impacts on some of the above schemes:

- The Joye and Michalevsky RSA exponent transform scheme reveals the private key if $e_p \leq N^{1/2}$ and $e_s < e_p N^{1/4}$ — that is, it is insecure precisely for the case of the small RSA exponent sizes allowed by HSMs that motivate their construction.
- An RSA user who generates a public exponent $e = e' + t\varphi(N)$ for $e' < N^{3/4}$ and $t \leq N^{1/2}$ reveals the factors of N , *even when d is large* and e' is unknown to the attacker. Implementations that attempt to apply exponent masking to RSA public exponents out of an abundance of caution to protect against Wiener’s attack or side-channel attacks may inadvertently reveal their private key if applied to the conventional practical construction of small public exponents.
- For the large, randomly generated public exponents as in [21, Section 5] and [1], our attacks give a previously undiscussed family of weak keys. These weak exponents are unlikely to arise for randomly generated instances, but a key generation algorithm may wish to verify that a particular key is not weak before publishing the public key.

We discuss these applications in detail in Sections 5 and 6.

Our results suggest that designers of alternative RSA constructions with large public exponents should apply these security analyses to avoid insecure parameters and exponent choices. None of the constructions we mention above contains an analysis of weak choices for their published public exponents.

2 Preliminaries

Notation and syntax. For an integer $z \in \mathbb{Z}$ we let $\lceil \log_2(z) \rceil$ be the number of bits in its representation as a binary string. For a set $\mathcal{X}$ we denote selection of a uniformly random element by $x \leftarrow_s \mathcal{X}$. To denote a range of integers we write $[a, b]$ to mean the set of integers between a and b , inclusive, or (a, b) to mean exclusive. We use $\perp$ to denote a special failure symbol.

For a modulus $M \in \mathbb{N}$ we let $\mathbb{Z}_M$ be the group consisting of $\{0, 1, \dots, (M-1)\}$ under addition modulo M . We let $\mathbb{Z}_M^*$ denote the group consisting of $\{z \in \mathbb{Z}_M \mid \gcd(z, M) = 1\}$ under multiplication modulo M .

The prime-counting function $\pi : \mathbb{N} \rightarrow \mathbb{N}$ returns the number of primes up to a given bound. On input $x \in \mathbb{N}$, $\pi(x)$ is the number of prime integers less than or equal to x . Our algorithm will not run the function π , and we use it only to bound probabilities.

RSA. For primes p, q and $N = pq$ recall that $\varphi(N) = (p-1)(q-1)$ and this is the order of the group $\mathbb{Z}_N^*$. In classic RSA [29], the public key includes the modulus N and an exponent $e \in \mathbb{Z}_{\varphi(N)}^*$ while the secret key includes an exponent $d \in \mathbb{Z}_{\varphi(N)}^*$ such that $ed = 1 \bmod \varphi(N)$. We will define all variables as needed but may informally refer to an RSA scheme’s “public e ” or “secret d .”

2.1 Related work on RSA cryptanalysis

RSA and related assumptions have undergone extensive cryptanalysis since its invention in [29]. A number of attacks use Coppersmith’s method for finding small roots of polynomials modulo integers via lattice reduction [11, 10].

Factoring with high bits known. Coppersmith gave an algorithm to factor an RSA modulus N when the most significant half of bits of one factor p are known [10]. In particular, we can phrase this as factoring when an approximation to p is known, with additive error at most $N^{1/4}$. Coppersmith’s original algorithm used lattice basis reduction to find small solutions to bivariate polynomials over the integers; Howgrave-Graham gave a reformulated algorithm as finding approximate integer common divisors [16]. We use this algorithm as a subroutine in our algorithm.

Theorem 2 (Bivariate Coppersmith). *Let $p(x, y) = \sum_{i,j} p_{ij}x^i y^j$ be a polynomial of degree at most δ in each separate variable x and y . Let X and Y be upper bounds on x and y respectively. Let $D = \max_{i,j} \{ |p_{ij}| X^i Y^j \}$. If*

$$(XY)^{3\delta/2} < D$$

then there exists a polynomial time algorithm to find (x_0, y_0) such that $p(x_0, y_0) = 0$, $x_0, y_0 \in \mathbb{Z}$, $x_0 < X$, $y_0 < Y$.

To find a small root modulo p , we will use May’s formulation [23] of the above Coppersmith and Howgrave-Graham results.

Theorem 3 (Coppersmith/Howgrave-Graham/May). *Let N be an integer of unknown factorization, which has a divisor $b \geq N^\beta$, $0 < \beta \leq 1$. Let $0 < \varepsilon \leq (1/7)\beta$. Furthermore, let $f(x)$ be a univariate monic polynomial of degree δ with at least one root modulo b , i.e. there exists at least one value of x_0 such that*

$$f(x_0) \equiv 0 \bmod b \text{ with } |x_0| \leq \frac{1}{2} N^{\beta^2/\delta-\varepsilon}.$$

Then, we can find all solutions for x_0 in polynomial time.

Attacks on small private exponent d . In 1990 Wiener [34] proposed an attack on the RSA cryptosystem for small d . He used continued fractions to give an algorithm to factor the RSA modulus N given an RSA public key (N, e) with corresponding secret exponent $d < N^{1/4}$. Boneh and Durfee [6] improved this bound to $d < N^{0.292}$ in 1999 using an algorithm based on Coppersmith’s method. Both methods consider the RSA equation $ed = 1 + k(N - (p + q) + 1)$ where d , k , and $p + q$ are unknown but bounded.

Attacks on partially known private exponent, with exponent masking. Some RSA-based schemes use a countermeasure known as exponent blinding or exponent masking, in which random multiples of $\varphi(N)$ are added to the private exponent d , typically to mitigate side channel attacks. Kocher [19] mentions this countermeasure in his work introducing timing attacks. In 2015, Cimoto, Mella and Susella gave an algorithm in [9] to factor an RSA modulus N in this exponent masking setting, given partial information about an exponent-masked private exponent $d^* = d + j\varphi(N)$. They improved upon 2012 algorithms described by Joye and Lepoint [17], which had covered the general case of a private exponent $d > \varphi(N)$. In 2022, Zhou, van de Pol, Yu, and Standaert [36] gave an algorithm targeting key recovery in this setting for masked private RSA-CRT coefficients. We consider the somewhat unusual question of what happens if an implementation masks a *public* exponent in Section 6.

Extensions of Wiener’s attack. In 2004 Blömer and May [5] gave a generalization of Wiener’s attack on short exponents d . For completeness, we give their full theorem.

Theorem 4 (Blömer-May). *Let $N \in \mathbb{N}$ be a product of two uniformly random primes p, q with $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil$. Then with high probability N can be factored in polynomial time given N and an $e \in \mathbb{Z}_{\varphi(N)}^*$ satisfying*

$$ex + y = 0 \pmod{\varphi(N)}$$

for some unknown $x, y \in \mathbb{Z}$ when x, y satisfy

$$0 < x \leq \frac{1}{3}N^{1/4} \tag{1}$$

$$y \leq cex \cdot N^{-3/4} \tag{2}$$

where the probability is over the primes p and q , and c is a constant.

Wiener’s attack is a special case of this attack where $y = -1$. This Blömer-May attack proceeds in three steps: first, they use continued fractions to recover the value of x , and then they use x to recover an approximation to p , and finally apply Coppersmith’s method to factor N given this partial information on p .

This is the most similar prior work to our attack; we observe that for our desired applications x is known, and thus we can obtain stronger bounds on y by re-analyzing their algorithm. We provide a detailed comparison in Section 4.

Further works have provided more exotic generalizations to even more special forms of e . In 2008 Nitaj [28] proposed another generalization of Wiener’s 2004

attack where the public key satisfies $ex - (p - u)(q - v)y = 1$ where $1 \leq y < x < 2^{-1/4}N^{1/4}$, $|u| < N^{1/4}$, $v = \lceil -qu/(p - u) \rceil$, and $(p - u)(q - v)$ is 10^{50} -smooth.

Deterministic equivalence between factoring and RSA private key recovery. Coppersmith's method has also been used by May [22] to give a deterministic polynomial-time algorithm to factor N given both public and private exponents (e, d) .

3 Factoring N from $\varphi(N)$ with noise

In our setting, the attacker learns an integer $a = y + k\varphi(N)$ where y and k are relatively small unknown integers. As usual, $\varphi(N)$ and the prime factors of N are also unknown. From the integer a (which can be seen as $\varphi(N)$ with noise) and the public modulus N , our attacker wishes to factor N . In the following, we assume that N is an RSA modulus with balanced primes, that is, that $N = pq$ with $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil$.

Notice that if more information than (a, N) is available to the attacker, then factoring may become easy:

- If the attacker learns $\varphi(N)$, they can write $p + q = \varphi(N) - N - 1$, learn $(p - q)^2 = (p + q)^2 - 4N$, and then solve for a factor p .
- If the attacker learns an exact multiple $a = k\varphi(N)$, they can solve for $k = \lceil a/N \rceil$, and then factor N as above.
- If the attacker learns both a and y such that $a = y + k\varphi(N)$, they can apply the above to $a - y$.

The above cases have long been noted [25]. This section concerns the case where the multiplier k and the noise y are unknown to the attacker, resisting the trivial attacks above. The following theorem shows how to factor N given $a = y + k\varphi(N)$ where both k and y are unknown but small.

Theorem 5. *Let $N \in \mathbb{N}$ be a product of two uniformly random primes p, q with $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil$. Then with high probability N can be factored in polynomial time given N and an $a \in \mathbb{Z}$ satisfying*

$$a \equiv y \pmod{\varphi(N)}$$

for some unknown $y \in \mathbb{Z}$ when a, y satisfy

$$\varphi(N) \leq a \leq N^{3/2} \tag{3}$$

$$y < a \cdot N^{-3/4} \tag{4}$$

where the probability is over the primes p and q .

The algorithm follows the second half of Blömer and May's algorithm [5], but since our setting and bounds are different, we re-analyze some steps. We prove Theorem 5 in the following sequence of lemmas, aligned with the steps of the algorithm. The proof concludes with Lemma 12.

Step 0: Take as input integers N and a . The algorithm begins by taking in the public information. Let $p > q$ be the larger factor of N . Lemma 6 below is a useful fact for the balanced-prime setting.

Lemma 6. When $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil$, $(3/2)\sqrt{N} \leq p + q \leq 3\sqrt{N}$.

Proof. In the balanced prime case, $p < 2q$ and $q \leq \sqrt{N} \leq p$. Therefore $\sqrt{N} \leq p \leq 2q \leq 2\sqrt{N}$ and $(1/2)\sqrt{N} \leq p/2 \leq q \leq \sqrt{N}$. Therefore $p + q \geq \sqrt{N} + (1/2)\sqrt{N} = (3/2)\sqrt{N}$ and $p + q \leq 2\sqrt{N} + \sqrt{N} = 3\sqrt{N}$.

Step 1: Learn k . Let k be the integer multiplier such that $a = y + k\varphi(N)$. Approximate k as $\lfloor a/N \rfloor$. We show (in Lemma 7) that k is at most an additive error of 6 from this approximation, so we can try all possible values.

Lemma 7. $0 \leq k - a/N < 6$.

Proof. For the first inequality, we have

$$k - \frac{a}{N} = \frac{a - y}{\varphi(N)} - \frac{a}{N} = \frac{a(p + q - 1) - yN}{\varphi(N)N}.$$

Since we have $p + q > \sqrt{N}$, the numerator will be positive when $y < aN^{-1/2}$. For the second inequality, we can bound $p + q \leq 3\sqrt{N}$ and $\varphi(N) > N/2$ to obtain

$$(p + q - 1)a - Ny < 3\sqrt{N}a < 3N^2 < 6N\varphi(N).$$

Dividing by $N\varphi(N)$ we have that $k - a/N < 6$.

Step 2: Approximate $p + q$. Let $s = p + q$. We can approximate $\tilde{s} = N + 1 - \lfloor a/k \rfloor$. The error will be $|s - \tilde{s}| \leq y/k$, as stated in Lemma 8.

Lemma 8. $(p + q) - \tilde{s} \leq y/k$.

Proof. This follows from the definition of $\tilde{s}$.

Step 3: Approximate $p - q$. We approximate $p - q$ as $\bar{s} = \lfloor \sqrt{\tilde{s}^2 - 4N} \rfloor$. The following three lemmas (9, 10, 11) are used to bound the error on this approximation; the final bound is stated in Lemma 11.

Lemma 9. For uniformly random prime integers p, q of the same size $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil = b$, and for a constant $0 \leq c \leq 1/4$, we have $|p - q| \geq c \cdot N^{1/2}$ with probability at least $1 - 4/b - 6c$. We assume $b > 16$.

The proof of Lemma 9 is more than a few lines, and is postponed to the Appendix. (While $b > 16$ is a necessary assumption for our proof, we consider factoring to already be practically easy for 32-bit moduli.)

Lemma 10. $y/k \leq N^{1/4}$.

Proof. $y \leq aN^{-3/4} - 1 \leq aN^{-3/4} - yN^{-3/4} \leq (a - y)N^{1/4}/\varphi(N) = kN^{1/4}$.

Lemma 11. $(p - q) - \bar{s} \leq (7/c) \cdot (y/k)$ with probability at least $1 - 4/b - 6c$, where $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil = b$.

Proof. We have

$$\begin{aligned} p - q - \sqrt{\bar{s}^2 - 4N} &= \frac{(p - q - \sqrt{\bar{s}^2 - 4N})(p - q + \sqrt{\bar{s}^2 - 4N})}{p - q + \sqrt{\bar{s}^2 - 4N}} \\ &= \frac{((p + q) - \bar{s})((p + q) + \bar{s})}{p - q + \sqrt{\bar{s}^2 - 4N}}. \end{aligned}$$

We have $(p + q) - \bar{s} = y/k$ from Lemma 8, $y/k \leq N^{1/4}$ from Lemma 10, and $|p - q| \geq cN^{1/2}$ from Lemma 9 (with probability at least $1 - 4/b - 6c$). Therefore we have

$$\frac{(p + q) + \bar{s}}{\sqrt{\bar{s}^2 - 4N} + (p - q)} \leq \frac{\bar{s} + p + q}{(p - q)} \leq \frac{N^{1/4} + 6N^{1/2}}{cN^{1/2}} \leq 7/c.$$

So the error on $(p - q)$ is at most $(7/c) \cdot (y/k)$, with probability at least $1 - 4/b - 6c$.

Step 4: Approximate $\tilde{p} = (\bar{s} + \bar{s})/2$ and apply Coppersmith/Howgrave-Graham to recover p from $\tilde{p}$ and N . When $|p - \tilde{p}| < N^{1/4}$ this method is guaranteed to recover p in polynomial time. Lemma 12 below proves that our approximation is within this $N^{1/4}$ bound, plus a constant factor.

Lemma 12. *The error on the approximation of p is $|p - \tilde{p}| \leq N^{1/4} \cdot (c + 7)/(2c)$ with probability at least $1 - 4/b - 6c$.*

Proof. We apply Lemmas 8 and 11 to obtain

$$|p - \tilde{p}| \leq \frac{1}{2} \left(\frac{7}{c} \cdot \frac{y}{k} + \frac{y}{k} \right) = \frac{c + 7}{2c} \frac{y}{k} \leq \frac{c + 7}{2c} N^{1/4}.$$

To reach the full Coppersmith bound without the constant factors, we brute force $\lceil \log_2((c + 7)/(2c)) \rceil$ bits of p . The success of Step 4 is then achieved by Coppersmith/Howgrave-Graham. The outcome is a recovery of p , meaning we have factored N .

This all occurs with probability at least $1 - 4/b - 6c$. We have $b = \lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil$, and can set $0 \leq c \leq 1/4$ to approach 0 so that the probability approaches 1 while the required number of bits to brute force is polynomial in $\log(1/c)$. Thus the algorithm succeeds with high probability, as claimed in Theorem 5.

To obtain a concrete bound, consider $b > 16$. Then our success probability is at least $0.75 - 6c$, and is at least $0.99 - 6c$ for practical (at least 1024-bit) moduli. The dependence on this $4/b$ term only reflects how primes are distributed at different bit sizes. $\square$

We implemented and ran the above algorithm on 2048-bit moduli and a range of a, y values, and report the results in Section 7. For the best Coppersmith lattice parameters, our attack factored N in over 98% of cases, where the probability

is over the uniformly random primes p, q . Even with brute-forcing a constant number of bits as in the remarks above, the largest cost was running LLL, which took at most seconds. The full details are in Section 7.

4 Comparison to alternate approaches

In this section, we explore related approaches to see whether other algorithmic tools can improve the bounds we obtain in Theorem 5.

4.1 Applying Blömer-May as a black box

An alternative approach to using Theorem 5 in the previous section is to apply Blömer-May's Theorem 4 as a black box. Note that since in our setting $a > \varphi(N)$ we cannot simply set their $e = a$ and $x = 1$, as this violates the requirement in Blömer-May that $e \in \mathbb{Z}_{\varphi(N)}^*$ and $e < \varphi(N)$. (The Blömer-May attack does not work when $e > \varphi(N)$.) If we are able to find two factors of a as $a = a_1 \cdot a_2$ such that $a_1 \in \mathbb{Z}_{\varphi(N)}^*$ and $a_2 \leq (1/3)N^{1/4}$ then the Blömer-May attack indeed applies as formulated in Theorem 4; we set $e = a_1$ and (known) $x = a_2$.

Unfortunately, even if we land in this case, the bound is worse than that of our attack in Section 3. Their bound results in $a = a_1 a_2 \leq (1/3)N^{5/4}$, while our theorem applies up to $a \leq N^{3/2}$. The bound on y relative to a is unchanged, but our re-analysis shows that it is possible to recover $y < N^{3/4}$ for maximal a , while simply applying their theorem results in a maximum value of $y < N^{1/2}$.

4.2 Bivariate Coppersmith

It is also tempting to try to improve Theorem 5 by applying bivariate Coppersmith's method.

After recovering k via elementary methods (Step 1 in the prior section), we can reformulate Steps 2–4 in our algorithm. We are solving for y in the equation $a - y = k\varphi(N)$ and can recast the problem as finding small roots of an integer bivariate polynomial, which is then solved via bivariate Coppersmith (Theorem 2). However this approach has strictly worse bounds than Theorem 5.

Let us rewrite $a - y = k\varphi(N)$ as a bivariate integer equation. Let $x = p + q$ be an unknown. Then we can rewrite the equation as

$$\begin{aligned} a - y &= k(N - x + 1) \\ kx - y + (a - kN - k) &= 0. \end{aligned}$$

We can apply Coppersmith's Theorem 2 to this integer equation. Let $f(x, y) = kx - y + (a - kN - k)$, and let X and Y be upper bounds on x and y respectively. Then $\delta = 1$. Since $p + q \leq 3N^{1/2}$, it follows that $X = 3N^{1/2}$. Now it remains to calculate D :

$$D = \max_{i,j} \{|p_{ij}| X^i Y^j\} = \max\{kX, Y, |a - kN - k|\}.$$

By Lemma 7, $k \approx a/N$, so kX is roughly equal to $3aN^{-1/2}$. $a - kN - k = k(p-1)(q-1) + y - kN - k = y - k(p+q)$, so $|a - kN - k| \approx a/\sqrt{N}$. Therefore, it follows that $D \approx \max\{Y, 3aN^{-1/2}\}$.

If $Y \geq 3aN^{-1/2}$, then $D = Y$, and $(XY)^{3/2} \geq Y$ meaning bivariate Coppersmith will fail. If $Y < 3aN^{-1/2}$, then $D = 3aN^{-1/2}$, and bivariate Coppersmith will succeed under the conditions

$$\begin{aligned} (XY)^{3\delta/2} &< D \\ (3N^{1/2}Y)^{3/2} &< 3aN^{-1/2} \\ Y &< a^{2/3}N^{-5/6}. \end{aligned}$$

This latter bound on Y (the upper bound on y) is strictly inferior to the bound $y < a \cdot N^{-3/4}$ of Theorem 5.

4.3 Primal and multivariate Coppersmith

We also explored approaches via heuristic multivariate Coppersmith methods to improve upon the bounds. In [30], Ryan surveys the literature on optimizing customized lattice constructions for multivariate Coppersmith-type problems, and creates an umbrella technique that is implemented in an automatic tool. We applied this tool, **cuso** (Coppersmith Solver), to solve the same integer bivariate Coppersmith problem discussed in the subsection above.

Our problem falls into the case of a linear bivariate equation. For this, **cuso** automatically attempts three methods to find a solution. These include a specialized linear solver which uses a primal Coppersmith construction, a Gröbner basis method, and a generalized multivariate Coppersmith method. In the following we discuss “recovering y ” but recall that for our setup, knowledge of y enables factoring N .

Empirically, we found that when used as a black box, **cuso** outperforms classic bivariate Coppersmith, but remains weaker than our approach. Table 1 gives **cuso**’s success rate for various sizes of a and y , compared to our algorithm, which performs strictly better. Let us explain here in more detail.

For concrete parameters of $\lceil \log_2(N) \rceil = 2048$ and $\lceil \log_2(a) \rceil = 3071$, the maximum size of y that **cuso** was able to recover is $\lceil \log_2(y) \rceil = 1023$. This outperforms classic bivariate Coppersmith, which, as shown in the previous subsection, can recover up to

$$\lceil \log_2(y) \rceil = (2/3) \cdot 3071 - (5/6) \cdot 2048 \approx 341.$$

However **cuso** is weaker than our approach; by Theorem 5 our theoretical maximum bound is

$$\lceil \log_2(y) \rceil = 3071 - (3/4) \cdot 2048 = 1535.$$

And as in Table 1, our approach works empirically up to $\lceil \log_2(y) \rceil = 1516$.

For concrete parameters of $\lceil \log_2(N) \rceil = 2048$ and $\lceil \log_2(a) \rceil = 2049$, we did not find a size of y that **cuso** could recover. However our approach was empirically able to recover y up to $\lceil \log_2(y) \rceil = 494$.

$\lceil \log_2(a) \rceil$	$\lceil \log_2(y) \rceil$	cuso Succ. %	Our Attack Succ. %
3071	512	1.00	1.00
3071	1023	0.94	1.00
3071	1323	0	0.98
3071	1516	0	0.98
2049	335	0	1.00
2049	494	0	1.00

Table 1. We ran the **cuso** solver on a variety of parameters using $\lceil \log_2(N) \rceil = 2048$. For each experiment, we chose the size of a and the size of y . Note that we time-out any attack that takes longer than 20 minutes and count it as a failure. (Our attack always took at most 13 seconds.) The “Succ. %” columns give the experimental success rate for recovering y and factoring N out of 50 trials.

We conjecture that this is due to steps in our algorithm that fall outside of generalizable Coppersmith techniques, such as Steps 2 and 3 in the proof of Theorem 5, but do not have a proven answer. We give a more extensive discussion of experimental results for our attack in Section 7.

5 Application to the Joye-Michalevsky transform

In 2018 Joye and Michalevsky (JM) presented a variant of RSA signatures intended to overcome hardware restrictions [18]. Their motivation is the following. Hardware Security Modules (HSMs) are a popular mechanism to securely store signing keys and efficiently handle high volumes of signature requests. Many HSMs only allow certain RSA public exponents, or disallow other values. For example, the Thales Luna 7 HSM only allows RSA public exponents 3, 17, and 65537¹, and only 65537 is allowed in FIPS mode. FIPS 186-5 [14] disallows most choices of public exponent: it specifies that “the exponent e shall be an odd positive integer such that $2^{16} < e < 2^{256}$ ”.

If the hardware disallows $e = 3$, but you would like to use it as your public exponent, is there a workaround? Joye and Michalevsky answered with a transform-signature scheme, which aims to do precisely this.

5.1 The JM transform-signature scheme

We begin by recalling the scheme in [18]. Scheme JM is not a standard signature scheme; we call it a *transform-signature scheme*. It consists of four algorithms which, in practice, are run by three different parties. The parties are the signer

¹ https://thalesdocs.com/gphsm/luna/7/docs/network/Content/PDF_Network/Utilities\%20Reference.pdf

```

JM.Kg( $e_p$ ):
1  $p, q \leftarrow \text{\$PrimesGen}$  ;  $N \leftarrow p \cdot q$  ;  $\varphi(N) \leftarrow (p-1)(q-1)$ 
2 If  $\gcd(e_p, \varphi(N)) \neq 1$  then restart
3  $t \leftarrow \text{\$}\{z \in \mathbb{Z} \mid 0 < z < N \text{ and } \gcd(z, \varphi(N)) = 1\}$ 
4  $e_s \leftarrow e_p \cdot t \bmod \varphi(N)$  s.t.  $e_s$  is supported on the HSM
5  $d_s \leftarrow e_s^{-1} \bmod \varphi(N)$ 
6  $sk \leftarrow (N, d_s)$  // Secret signing key
7  $tk \leftarrow (N, t)$  // Public transform key
8  $vk \leftarrow (N, e_p)$  // Public verification key
9 Return  $(sk, tk, vk)$ 

JM.Sign( $sk, m$ ):
10  $(N, d_s) \leftarrow sk$ 
11  $\sigma' \leftarrow \text{\$H}(m)^{d_s} \bmod N$ 
12 Return  $\sigma'$ 

JM.Transform( $tk, \sigma'$ ):
13  $(N, t) \leftarrow tk$ 
14  $\sigma \leftarrow (\sigma')^t \bmod N$ 
15 Return  $\sigma$ 

JM.Verify( $vk, m, \sigma$ ):
16  $(N, e_p) \leftarrow vk$ 
17 Return true iff  $\sigma^{e_p} = \text{H}(m) \bmod N$ 

```

Fig. 1. The transform-signature scheme JM. Line 4, where e_s is generated but must be supported on the HSM, could be implemented in different ways depending on the particular restrictions; we discuss this in the body.

(the HSM), the transformer (the operator of the HSM) and a verifier. Now scheme JM consists of algorithms:

- **JM.Kg:** This takes as input a desired public exponent $e_p \in \mathbb{N}$ and returns keys sk, tk, vk for the signer, transformer and verifier, respectively. Only the signing key sk is secret; both tk and vk are public. In brief, the signing key is a secret exponent $d_s \in \mathbb{Z}_{\varphi(N)}^*$ with an associated but unused secret $e_s = d_s^{-1} \bmod \varphi(N)$. The verification key is e_p as desired from the input. The transform key is $t \in \mathbb{Z}_{\varphi(N)}^*$ such that $e_s = e_p \cdot t \bmod \varphi(N)$.
- **JM.Sign:** This operates as a usual RSA signing algorithm. It takes as input $sk = (N, d_s)$ and a message $m \in \{0, 1\}^*$ to return a signature $\sigma' \in \mathbb{Z}_N^*$. The signature is $\text{H}(m)^{d_s} \bmod N$ for a full-domain hash function $\text{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_N^*$.
- **JM.Transform:** This takes as input the transform key $tk = (N, t)$ and a signature $\sigma' \in \mathbb{Z}_N^*$ to return a transformed signature $\sigma \in \mathbb{Z}_N^*$. The transformation specifies that $\sigma = (\sigma')^t \bmod N$.
- **JM.Verify:** This is usual RSA verification, taking as input $vk = (N, e_p)$ and verifying whether the input σ and m satisfy $\sigma^{e_p} = \text{H}(m) \bmod N$.

We provide the scheme JM in more detail in Figure 1. We do so for completeness but in fact, the following attack only considers the key-generation method JM.Kg. Note that one subtlety arises with the selection of t and e_s . Key generation must output values such that $e_s = e_p \cdot t$ and e_s is “supported on the HSM.” If a large enough fraction of exponents are allowed, then (re-)sampling t until e_s is an allowed exponent is efficient. If the HSM has a very restrictive set of allowed exponents, then e_s can be sampled and t determined from e_s and e_p . However any implementation choices here would modify the distribution of possible exponents; in particular security of the scheme relies on high-entropy e_s .

5.2 Attacking small parameter regimes

The transform-signature scheme JM produces public information (N, e_p, t) and secret information e_s . There is additional secret information including the factors p, q and $\varphi(N), d_s$. Our problem statement is the following:

Given (N, e_p, t) , under what conditions can you efficiently factor N ?

Recovering p, q is the strongest attack one can hope for, as it immediately yields the secret keys and all associated values.

Joye and Michalevsky observed that e_s must remain secret, else one can recover the secret key from the relationship between e_p and e_s . Our attack strengthens this observation: even if e_s is secret and drawn from a cryptographically large set, we can recover the secret key from the public key and transform key if the exponents e_p and e_s are small relative to N . In particular, the attack of Section 3 applies when $e_p \leq N^{1/2}$ and $e_s < e_p N^{1/4} < N^{3/4}$. This is detailed in the below corollary, which is a result of Theorem 5, as well as experimental results we conducted in Section 7.

Unfortunately, this means that the Joye-Michalevsky scheme is insecure in precisely the case for which it was designed, to, for example, generate signatures for $e_p = 3$ using an HSM that only supports RSA exponents allowed by FIPS 186-5, so that $2^{16} < e_s < 2^{256}$. For a 2048-bit modulus N , and $e_p = 3$, our attack would apply up to a 512-bit e_s .

Our attack suggests that care must be taken with parameter choices, and moreover that the usual RSA recommendations are necessary but not sufficient for this transform setting.

Corollary 13. *Let public (N, e_p, t) and secret e_s be generated by the Joye-Michalevsky transform, as described above. If $t \geq N^{1-\varepsilon}$ for $0 < \varepsilon \leq 1$ and if*

$$N^\varepsilon \leq e_p \leq N^{1/2} \tag{5}$$

$$e_s < e_p N^{1/4-\varepsilon} \tag{6}$$

then with high probability N can be factored in polynomial time.

Before presenting the proof, let us remark on the variable ε . In the transform, t is selected as $t \leftarrow_s [1, (N-1)]$ subject to $\gcd(t, \varphi(N)) = 1$. With high probability $\lceil \log_2(t) \rceil \approx \lceil \log_2(N) \rceil$ and ε is very small.

Proof (Corollary 13). To prove the corollary we need only translate e_p, e_s, t into a, y of Theorem 5. We set $a = e_p \cdot t$ and $y = e_s$. First, if $e_p \leq N^{1/2}$ then $e_p t < N^{3/2}$ since $t < N$. Moreover we have $e_p \geq N^\varepsilon \geq N/N^{1-\varepsilon} \geq \varphi(N)/t$ so $e_p t \geq \varphi(N)$. Together these are exactly the first condition of Theorem 5; that is, that $\varphi(N) \leq a \leq N^{3/2}$ for $a = e_p t$.

Moving to the second condition, if $e_s < e_p N^{1/4-\varepsilon}$ then $e_s < e_p N^{1-\varepsilon-3/4} < e_p t N^{-3/4}$. This is precisely the second condition of Theorem 5, which asks that $y < a N^{-3/4}$. Thus we can apply that theorem with $a = e_p \cdot t$ and $y = e_s$, which now says that with high probability N can be factored in polynomial time. $\square$

5.3 Discussion

We have shown that the JM scheme can be attacked in polynomial time when e_p and e_s are small. We conclude our discussion of this application with three remarks. Joye and Michalevsky proved that their scheme satisfies existential unforgeability under a chosen-message attack (UF-CMA). First, to our knowledge, the security proof of [18] is not necessarily violated. The proof argues that if there is an efficient adversary violating UF-CMA (for a transform signature scheme) of JM then one can construct an efficient adversary violating the one-wayness of RSA. These hardness assumptions should depend on the distribution of keys output by an RSA key generator. When weak keys are generated, the proof argument still holds but is vacuous: there is already an efficient adversary violating the one-wayness of RSA. This is a reminder of the importance of considering, or at least parameterizing by, distributions of RSA keys in proofs.

Second, a simple countermeasure to the attack is to select e_p, e_s which are not vulnerable to Corollary 13. One or both must then be large. Unfortunately, from a practical perspective this means that either signature verification will be much slower (if e_p is large) or e_s may be unsupported by HSMs or other signing implementations that limit the size of allowed public exponents.

Third, one could modify the JM scheme in potentially useful ways. For example, given the use case, it seems that t could remain secret to the HSM operator. If t is secret, our attack doesn't apply, but Blömer-May will, where t translates to the unknown x , e_s to the unknown y , and e_p to the public e in Theorem 4. The Blömer-May attack succeeds if $t \leq (1/3)N^{1/4}$ and $e_s \leq cN^{-3/4}e_p t$. In any case, we reiterate that care should be taken when transforming RSA exponents.

6 Application to e larger than $\varphi(N)$

Recall that the attack of Section 3 applies when some $a \geq \varphi(N)$ is known to the attacker. A natural setting where this could arise is schemes that select the RSA public exponent e to be larger than $\varphi(N)$. This does not immediately lead to an attack. However it shows that there is a class of weak choices of e , in particular those such that $e \bmod \varphi(N)$ is small. We discuss applications where this may occur.

RSA with public metadata. One example of such a scheme is the RSA-with-public-metadata scheme of [1], published at PETS 2025 and motivating a recent IRTF draft [2]. The main scheme guarantees that $e < \varphi(N)$, but a variant with better concrete security [1, Appendix C] leads to potentially larger e . There, a public key is set to be $e \leftarrow e' \cdot \text{H}(\text{data})$ for some specified metadata. The hash function maps to elements of size at most $\varphi(N)^{1/2}$. This is not immediately obvious so we explain. The authors prove, in their Lemma 1, that an output h of the hash function satisfies $h < p'$ and $h < q'$, where $\varphi(N) = 4p'q'$. It follows that $4h^2 < 4p'q'$ so $h < 1/2 \cdot \varphi(N)^{1/2}$.

If e' is full-size, or at least larger than $N^{1/2}$, it is thus possible to have $\varphi(N) < e < N^{3/2}$. Our attack applies in these parameter regimes if $e \bmod \varphi(N)$ is small, meaning $e \bmod \varphi(N) < N^{3/4}$. Note that one would need to be very unlucky to select such a weak key: for the most extreme parameters this happens with probability about $N^{-1/4}$. Nonetheless, it is a previously undescribed class of weak keys. There is no randomness in the selection of $e \leftarrow e' \cdot \text{H}(\text{data})$, but the scheme could check and re-sample e if it is weak (for example, with a salt). Or, the exponent e' could also be specified to be small, with $e' < \varphi(N)^{1/2}$, so the product remains $e < \varphi(N)$.

Other schemes with large e . The aggregate signature scheme of Lysanskaya, Micali, Reyzin, and Shacham [21, Section 5] is another example of large RSA exponents appearing in the literature. The scheme “require[s] e to be a prime larger than N ”. Thus as above, we exhibit a class of previously un-analyzed weak keys for this construction.

Interestingly, there are schemes [8] which use large e but specifically require $e = N$. This avoids our attack because $N \bmod \varphi(N)$ is always about the size of $N^{1/2}$ so Eq. (4) of Theorem 5 does not hold. Intuitively, setting $e = N$ leaks no information about $\varphi(N)$.

Wiener’s attack and countermeasure. In 1989, Wiener proposed an attack against RSA with a small private exponent d [34]. As a countermeasure, he suggested adding multiples of $\text{lcm}(p-1, q-1)$: “In order to make k larger, one must make the public exponent e larger... This can be done by adding a multiple of $\text{lcm}(p-1, q-1)$ to e .” He goes on to argue that $e > N^{3/2}$ mitigates the continued fraction attack. Our work shows that generating an exponent e of this form may be vulnerable *even if d is full sized*. Consider an unwitting implementer who generates a relatively small e and a corresponding full-sized d as a starting point, but decides to apply Wiener’s countermeasure and generate a large e out of an overabundance of caution.

In that case, the public exponent is $e' = e + j \cdot \text{lcm}(p-1, q-1)$ for some integer j and some e that is unknown to the attacker. Then we can let $a = \text{gcd}(p-1, q-1)e' = \text{gcd}(p-1, q-1)e + j(p-1)(q-1)$ and $y = \text{gcd}(p-1, q-1)e$. If $\text{gcd}(p-1, q-1)e' < N^{3/2}$ and $e < e' \cdot N^{-3/4}$, then our attack will successfully factor N . Note that our attack requires $e' < N^{3/2}$, and thus does not apply for Wiener’s recommended exponents of size $e > N^{3/2}$.

Exponent masking. Kocher’s paper introducing cryptographic timing attacks [19] suggests exponent masking as a potential countermeasure: “If anonymity is important or if further masking is required, a random multiple of $\varphi(N)$ can be added to the exponent before each modular exponentiation.” While this is intended to be applied to the private exponent d , and typically operations on e wouldn’t need to be protected (and an implementation operating on e likely doesn’t have access to the private key to mask anyway) an excessively cautious implementation that attempts to protect e by ensuring an attacker only has access to a blinded version may inadvertently introduce a vulnerability. Interestingly, this would also allow for key recovery even if the unmasked value of e is unknown, the recovered masked e' contains arbitrary noise in the least significant bits, or the attacker only obtains most significant bits of the recovered masked e' .

Let $e' = e + j\varphi(N)$ for some unknown j and unknown small e , and suppose that the attacker has observed the value of e' either openly in a blinded/masked public key, or via a side-channel attack on a blinded/masked implementation. Then we can set $a = e'$ and $y = e$. If $\varphi(N) < e' < N^{3/2}$ and $e < e'N^{-3/4}$, then our attack will succeed and we will be able to factor N successfully.

7 Experimental results and optimizations

We implemented our attack in Sage [33] to experimentally validate our bounds and gather timing data. We used the `flatter` algorithm of Ryan and Heninger [31] for lattice reduction. We ran our experiments on a computer with a 2.40GHz Intel Xeon E5-2673 processor and 32GB RAM, but for the parameters we report the experiments also run in seconds on a laptop.

7.1 Details on applying Coppersmith/Howgrave-Graham

In Step 4 of our algorithm, we apply the Coppersmith/Howgrave-Graham approach [23] to recover a factor p of N from an approximation $\tilde{p}$. In theory, this algorithm runs in polynomial time to recover $N^{1/4}$ bits of p . However, actually implementing this algorithm requires choosing concrete parameters.

In Coppersmith’s algorithm, given an upper bound $X > x$, our goal is to construct a polynomial h such that $h \equiv 0 \pmod{p}$, and $|h(xX)| < N^\beta = p \approx N^{1/2}$.

In May’s algorithm, the lattice L that is constructed depends on two parameters: the multiplicity m and the maximum degree shift l . The algorithm’s success can be predicted by verifying that LLL can recover a lattice vector shorter than the bound required by the algorithm. We heuristically assumed we could achieve LLL approximation factor $1.02^{\dim L}$, where $\dim L$ is the dimension of L , following Nguyen and Stehlé’s experiments [27]. For given values of m and l it is straightforward to compute the determinant and dimension of L .

Given this heuristic assumption, the LLL algorithm guarantees that

$$|h(xX)| \leq 1.02^{\dim(L)} \det(L)^{1/\dim(L)}$$

where $\dim L$ is the dimension of L and $\det(L)$ is the determinant of L . Therefore, if we choose parameters m and l such that

$$1.02^{\dim(L)} \det(L)^{1/\dim(L)} \leq N^{1/2}$$

then we will have

$$|h(xX)| \leq N^{1/2}$$

meeting the sufficient condition for Coppersmith algorithm's to succeed. We computed optimal X bounds for various lattice sizes by ranging over possible values for m and for l and choosing the maximal X for each multiplicity m . This $\lceil \log_2(X) \rceil$ is included as a column in the coming Table 2.

7.2 Experimental results

We summarize our results in Table 2. At a high level, they validate the parameters for which Theorem 5 implies we can factor N with high probability. To see this, consider the columns in Table 1 which report the size of a ($\lceil \log_2(a) \rceil$), the theoretical maximum y we can solve for ($\lceil \log_2(y) \rceil$) and the success rate (the fraction of experiments where we did recover y and factor N). This maximum y is coming from our statement of Theorem 5; as long as $a \leq N^{3/2}$ we expect $y < a \cdot N^{-3/4}$. The theorem is proved “with high probability, where the probability is over the primes p and q .” This is what we aim to experimentally validate. Our success rates for different parameter choices range from 0.88 to 1, justifying this claim of high probability.

In the remainder of this section we explain the experiments, and other intermediate data, in more detail.

7.3 Experimental details

We ran our experiments on 2048-bit N ; the bounds are relative so they scale arbitrarily. We chose representative values of a at each end of our theoretical bounds. For each value of a , we chose a range of values of m , ranging from tiny to medium-sized lattices, and computed the maximal bound X on $p - \tilde{p}$ that the algorithm could tolerate in theory with those parameters.

We used Lemma 9 to choose a c that should result in $|p - q| \geq cN^{1/2}$ with probability ≥ 0.9 . We set this value to $c = 1/40$.

Once we had $\log_2(X)$ and a chosen value of c , we applied Lemma 11 to obtain $|p - \tilde{p}| \leq (c + 7)/(2c) \cdot y/k \approx 140 \cdot yN/a$. This gives the theoretical maximum size of y that could be recovered with success rate ≥ 0.9 . This is included in Table 2 as the column “ $\lceil \log_2(y) \rceil$ ”.

We then experimentally verified these theoretical results by running the attack for our chosen parameters. For each parameter set in the table, we ran the experiment 50 times and compared our empirical success rate to the theoretical success rate (0.9).

$\lceil \log_2(a) \rceil$	m	l	$\dim L$	$\lceil \log_2(y) \rceil$	Succ. %	$\lceil \log_2(X) \rceil$	$k - \lfloor \frac{a}{N} \rfloor$	$\frac{ p-\tilde{p} }{(y/k)}$	LLL time
3071	1	4	5	1323	0.98	307	1.82	8.04	0.035s
3071	2	5	7	1406	0.96	389	1.90	7.01	0.055s
3071	3	6	9	1443	1.00	426	1.86	6.07	0.079s
3071	5	8	13	1476	0.96	459	1.94	6.20	0.192s
3071	10	13	23	1402	0.92	485	1.90	6.13	0.955s
3071	22	25	47	1516	0.98	500	1.74	6.80	12.068s
2049	1	4	5	335	1.00	341	1.00	6.12	0.032s
2049	2	5	7	384	0.88	389	1.00	8.34	0.056s
2049	3	6	9	421	0.98	426	1.00	5.73	0.079s
2049	5	8	13	454	1.00	459	1.00	6.02	0.190s
2049	10	13	23	480	0.96	485	1.00	7.72	0.956s
2049	22	25	47	494	0.98	500	1.00	7.41	12.059s

Table 2. We implemented and ran our attack on a variety of parameters using $\lceil \log_2(N) \rceil = 2048$. For each experiment, we chose the size of a and the multiplicity and shift parameters (m and l) for Coppersmith. From these, we computed the dimension of the lattice constructed in Coppersmith ($\dim L = m + l$). The next two columns are perhaps the most important: $\lceil \log_2(y) \rceil$ gives the theoretical size of y for which Theorem 5 implies a 90% chance of successfully factoring N . The “Succ. %” column gives the experimental success rate for this value out of 50 trials. The remaining columns include intermediate calculations in the attack, which are explained in the body of this section, as well as the lattice reduction time.

We also computed the ratio $|p - \tilde{p}|/(y/k)$ empirically, to measure the distance between our theoretical bound of $|p - \tilde{p}| \approx 140 \cdot y/k$ above to the observed size of $|p - \tilde{p}|$ in our experiments. We calculated the ratio by taking the median over 50 experiments for each choice of parameters and included it in the second-to-last column of Table 2.

In addition to experiments simulating the whole attack, we also ran experiments on approximating k . For each choice of N , a and y sizes that were in the table, we generated 50 sets of values for N , a , y , k , and then calculated $k - \lfloor a/N \rfloor$ for each set, and took the mean of these differences. We included this mean error on the approximation of k in the table, as the column “ $k - \lfloor a/N \rfloor$ ”.

Outperforming theoretical guarantees. Notice that our experiments outperformed our theoretically guaranteed success rate of 0.9. With the data in the table, we see two reasons for this. First, the ratio $|p - \tilde{p}|/(y/k)$ was lower than expected. Initially, we had an upper bound $|p - \tilde{p}| \lesssim 140 \cdot y/k$, assuming $|p - q| \geq (1/40)\sqrt{N}$. However, as the table suggests, it appears to be the case that $|p - \tilde{p}| \lesssim 10(y/k)$. The theoretical bounds are thus a bit conservative on the size of y that can be recovered.

Second, we also found that, for our choices of the sizes of N , a , y , the value $k - \lfloor a/N \rfloor$ was smaller on average than the theoretical upper bound, which we

calculated to be 7. In fact, in all the experiments on k we performed, the value $k - \lfloor a/N \rfloor$ never exceeded 3. This verifies that our theoretical result $k - a/N \leq 6$ is conservative as an upper bound.

8 Conclusion

In this paper, we observe an apparently previously un-analyzed source of weak keys for RSA: large public exponents that are close to a small multiple of $\varphi(N)$. While such exponents are unlikely to arise through random chance, they do arise through exponent transforms and masking schemes.

The methods we apply in our attack are all quite elementary (except for the use of univariate Coppersmith’s theorem); we ourselves were surprised that they apparently could not be outperformed by custom multivariate Coppersmith-type algorithms. We leave it to future work to improve further upon these bounds.

Acknowledgments

We thank Marc Joye and Yan Michalevsky for their feedback and suggestions, and Keegan Ryan for conversations about Coppersmith’s method. Shea was supported by NSF grants CNS-2048563, CNS-1513671, and CNS-2154272, and an FMA fellowship award from Qualcomm. Heninger was supported by a gift from Google and NSF grant DMS-2411704.

References

1. Amjad, G., Yeo, K., Yung, M.: RSA blind signatures with public metadata. In: PETS 2025 (2025), <https://eprint.iacr.org/2023/1199> (Cited on 2, 3, 15.)
2. Amjad, G.A., Hendrickson, S., Wood, C.A., Yeo, K.W.L.: Partially blind RSA signatures (2025), <https://datatracker.ietf.org/doc/draft-irtf-cfrg-partially-blind-rsa> (Cited on 2, 15.)
3. Aono, Y.: A new lattice construction for partial key exposure attack for RSA. In: Jarecki, S., Tsudik, G. (eds.) PKC 2009. LNCS, vol. 5443, pp. 34–53. Springer, Berlin, Heidelberg (Mar 2009). https://doi.org/10.1007/978-3-642-00468-1_3 (Cited on 3.)
4. Blömer, J., May, A.: New partial key exposure attacks on RSA. In: Boneh, D. (ed.) CRYPTO 2003. LNCS, vol. 2729, pp. 27–43. Springer, Berlin, Heidelberg (Aug 2003). https://doi.org/10.1007/978-3-540-45146-4_2 (Cited on 3.)
5. Blömer, J., May, A.: A generalized wiener attack on RSA. In: Bao, F., Deng, R., Zhou, J. (eds.) PKC 2004. LNCS, vol. 2947, pp. 1–13. Springer, Berlin, Heidelberg (Mar 2004). https://doi.org/10.1007/978-3-540-24632-9_1 (Cited on 3, 5, 6.)
6. Boneh, D., Durfee, G.: Cryptanalysis of RSA with private key d less than $N^{0.292}$. In: Stern, J. (ed.) EUROCRYPT’99. LNCS, vol. 1592, pp. 1–11. Springer, Berlin, Heidelberg (May 1999). https://doi.org/10.1007/3-540-48910-X_1 (Cited on 2, 5.)

7. Boneh, D., Durfee, G., Frankel, Y.: An attack on RSA given a small fraction of the private key bits. In: Ohta, K., Pei, D. (eds.) ASIACRYPT'98. LNCS, vol. 1514, pp. 25–34. Springer, Berlin, Heidelberg (Oct 1998). https://doi.org/10.1007/3-540-49649-1_3 (Cited on 3.)
8. Catalano, D., Gennaro, R., Howgrave-Graham, N., Nguyen, P.Q.: Paillier's cryptosystem revisited. In: Proceedings of the 8th ACM Conference on Computer and Communications Security. p. 206–214. CCS '01, Association for Computing Machinery, New York, NY, USA (2001). <https://doi.org/10.1145/501983.502012>, <https://doi.org/10.1145/501983.502012> (Cited on 15.)
9. Cimato, S., Mella, S., Susella, R.: New results for partial key exposure on RSA with exponent blinding. In: 2015 12th International Joint Conference on e-Business and Telecommunications (ICETE). vol. 04, pp. 136–147 (2015) (Cited on 3, 5.)
10. Coppersmith, D.: Finding a small root of a bivariate integer equation; factoring with high bits known. In: Maurer, U.M. (ed.) EUROCRYPT'96. LNCS, vol. 1070, pp. 178–189. Springer, Berlin, Heidelberg (May 1996). https://doi.org/10.1007/3-540-68339-9_16 (Cited on 3, 4.)
11. Coppersmith, D.: Finding a small root of a univariate modular equation. In: Maurer, U.M. (ed.) EUROCRYPT'96. LNCS, vol. 1070, pp. 155–165. Springer, Berlin, Heidelberg (May 1996). https://doi.org/10.1007/3-540-68339-9_14 (Cited on 2, 3, 4.)
12. Dusart, P.: Estimates of some functions over primes without R.H. (2010), <https://arxiv.org/abs/1002.0442> (Cited on 22.)
13. Ernst, M., Jochemsz, E., May, A., de Weger, B.: Partial key exposure attacks on RSA up to full size exponents. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 371–386. Springer, Berlin, Heidelberg (May 2005). https://doi.org/10.1007/11426639_22 (Cited on 3.)
14. FIPS 186-5: Digital signature standard (2023), <https://csrc.nist.gov/pubs/fips/186-5/final> (Cited on 1, 11.)
15. Howgrave-Graham, N.: Finding small roots of univariate modular equations revisited. In: Darnell, M. (ed.) 6th IMA International Conference on Cryptography and Coding. LNCS, vol. 1355, pp. 131–142. Springer, Berlin, Heidelberg (Dec 1997). <https://doi.org/10.1007/bfb0024458> (Cited on 3.)
16. Howgrave-Graham, N.: Approximate integer common divisors. In: International cryptography and lattices conference. pp. 51–66. Springer (2001) (Cited on 4.)
17. Joye, M., Lepoint, T.: Partial key exposure on RSA with private exponents larger than N . In: Ryan, M.D., Smyth, B., Wang, G. (eds.) Information Security Practice and Experience. pp. 369–380. Springer Berlin Heidelberg, Berlin, Heidelberg (2012) (Cited on 5.)
18. Joye, M., Michalevsky, Y.: RSA signatures under hardware restrictions. In: Proceedings of the 2018 Workshop on Attacks and Solutions in Hardware Security. p. 51–54. ASHES '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3266444.3266451>, <https://doi.org/10.1145/3266444.3266451> (Cited on 2, 11, 14.)
19. Kocher, P.C.: Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In: Koblitz, N. (ed.) CRYPTO'96. LNCS, vol. 1109, pp. 104–113. Springer, Berlin, Heidelberg (Aug 1996). https://doi.org/10.1007/3-540-68697-5_9 (Cited on 2, 5, 16.)
20. Lu, Y., Zhang, R., Lin, D.: New partial key exposure attacks on CRT-RSA with large public exponents. In: Boureanu, I., Owesarski, P., Vaudenay, S. (eds.)

- ACNS 14 International Conference on Applied Cryptography and Network Security. LNCS, vol. 8479, pp. 151–162. Springer, Cham (Jun 2014). https://doi.org/10.1007/978-3-319-07536-5_10 (Cited on 3.)
21. Lysyanskaya, A., Micali, S., Reyzin, L., Shacham, H.: Sequential aggregate signatures from trapdoor permutations. In: Cachin, C., Camenisch, J. (eds.) EU-ROCRYPT 2004. LNCS, vol. 3027, pp. 74–90. Springer, Berlin, Heidelberg (May 2004). https://doi.org/10.1007/978-3-540-24676-3_5 (Cited on 2, 3, 15.)
 22. May, A.: Computing the RSA secret key is deterministic polynomial time equivalent to factoring. In: Franklin, M. (ed.) CRYPTO 2004. LNCS, vol. 3152, pp. 213–219. Springer, Berlin, Heidelberg (Aug 2004). https://doi.org/10.1007/978-3-540-28628-8_13 (Cited on 6.)
 23. May, A.: Using LLL-reduction for solving RSA and factorization problems. In: Nguyen, P.Q., Vallée, B. (eds.) The LLL Algorithm: Survey and Applications, pp. 315–348. Springer Berlin Heidelberg, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-02295-1_10, https://doi.org/10.1007/978-3-642-02295-1_10 (Cited on 4, 16.)
 24. May, A., Nowakowski, J., Sarkar, S.: Partial key exposure attack on short secret exponent CRT-RSA. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part I. LNCS, vol. 13090, pp. 99–129. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92062-3_4 (Cited on 3.)
 25. Miller, G.L.: Riemann’s hypothesis and tests for primality. *Journal of Computer and System Sciences* **13**(3), 300–317 (1976). [https://doi.org/https://doi.org/10.1016/S0022-0000\(76\)80043-8](https://doi.org/https://doi.org/10.1016/S0022-0000(76)80043-8), <https://www.sciencedirect.com/science/article/pii/S0022000076800438> (Cited on 6.)
 26. Moody, D., Perlner, R., Regenscheid, A., Robinson, A., Cooper, D.: NIST transition to post-quantum cryptography standards (2024), <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf> (Cited on 1.)
 27. Nguyen, P.Q., Stehlé, D.: LLL on the average. In: Hess, F., Pauli, S., Pohst, M. (eds.) *Algorithmic Number Theory*. pp. 238–256. Springer Berlin Heidelberg, Berlin, Heidelberg (2006) (Cited on 16.)
 28. Nitaj, A.: Another generalization of wiener’s attack on RSA. In: Vaudenay, S. (ed.) *Progress in Cryptology – AFRICACRYPT 2008*. pp. 174–190. Springer Berlin Heidelberg, Berlin, Heidelberg (2008) (Cited on 5.)
 29. Rivest, R.L., Shamir, A., Adleman, L.M.: A method for obtaining digital signatures and public-key cryptosystems. *Communications of the Association for Computing Machinery* **21**(2), 120–126 (Feb 1978). <https://doi.org/10.1145/359340.359342> (Cited on 4.)
 30. Ryan, K.: Solving multivariate Coppersmith problems with known moduli. In: EUROCRYPT 2025 (2025) (Cited on 10.)
 31. Ryan, K., Heninger, N.: Fast practical lattice reduction through iterated compression. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part III. LNCS, vol. 14083, pp. 3–36. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_1 (Cited on 16.)
 32. Sarkar, S., Maitra, S.: Partial key exposure attack on CRT-RSA. In: Abdalla, M., Pointcheval, D., Fouque, P.A., Vergnaud, D. (eds.) ACNS 09 International Conference on Applied Cryptography and Network Security. LNCS, vol. 5536, pp. 473–484. Springer, Berlin, Heidelberg (Jun 2009). https://doi.org/10.1007/978-3-642-01957-9_29 (Cited on 3.)
 33. The Sage Developers: SageMath, the Sage Mathematics Software System (Version 9.5) (2022), <https://www.sagemath.org> (Cited on 16.)

34. Wiener, M.J.: Cryptanalysis of short RSA secret exponents (abstract). In: Quisquater, J.J., Vandewalle, J. (eds.) EUROCRYPT'89. LNCS, vol. 434, p. 372. Springer, Berlin, Heidelberg (Apr 1990). https://doi.org/10.1007/3-540-46885-4_36 (Cited on 5, 15.)
35. Wiener, M.: Cryptanalysis of short RSA secret exponents. IEEE Transactions on Information Theory **36**(3), 553–558 (1990). <https://doi.org/10.1109/18.54902> (Cited on 2.)
36. Zhou, Y., van de Pol, J., Yu, Y., Standaert, F.X.: A third is all you need: Extended partial key exposure attack on CRT-RSA with additive exponent blinding. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part IV. LNCS, vol. 13794, pp. 508–536. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22972-5_18 (Cited on 3, 5.)

Appendix

This appendix gives the proof of Lemma 9 which was introduced in Section 3.

Lemma 9. *For uniformly random prime integers p, q of the same size $\lceil \log_2(p) \rceil = \lceil \log_2(q) \rceil = b$, and for a constant $0 \leq c \leq 1/4$, we have $|p - q| \geq c \cdot N^{1/2}$ with probability at least $1 - 4/b - 6c$. We assume $b > 16$.*

Before proceeding to the proof, we remark that $b > 16$ is needed to apply Dusart's non-asymptotic statements of the prime number theorem [12]. The precise bound used by Dusart is $N \geq 60,184$, which is achieved if each prime factor of N is over 16 bits. If $b \leq 16$ we consider factoring to be (practically) easy.

Proof. We use the fact that the prime-counting function π can be bounded as

$$\frac{x}{\ln(x) - 1} \leq \pi(x) \leq \frac{x}{\ln(x) - 1.1}$$

for $x \geq 60184$ (or $\lceil \log_2(x) \rceil > 16$) as proved by Dusart [12]. From this fact, we give a lower bound on the total number of primes in the interval $(2^{b-1}, 2^b)$; we call this quantity D . For any prime p in the interval, we also give an upper bound on the number of primes q such that $|p - q| < c \cdot N^{1/2}$; we call this quantity Q . In particular we will prove that:

$$D = \pi(2^b) - \pi(2^{b-1}) \geq \log_2(e) 2^{b-1} \left(\frac{b - 3.8}{(b - 1.4)(b - 2.6)} \right) \quad (7)$$

$$Q = \pi(p + c \cdot N^{1/2}) - \pi(p - c \cdot N^{1/2}) \leq \log_2(e) 2^{b-1} \left(\frac{4bc + 2.4}{(b - 1.4)(b - 2.6)} \right) \quad (8)$$

From these two equations, we bound the probability that q is selected to be close to p . That is, for random and independent primes p, q the above two equations imply

$$\Pr[|p - q| < c \cdot N^{1/2}] \leq \frac{Q}{D} \leq \frac{4bc + 2.4}{b - 3.8} < \frac{4bc + 2.4}{0.75 \cdot b} < 6c + 4/b$$

and thus that

$$\Pr[|p - q| \geq c \cdot N^{1/2}] \geq 1 - 4/b - 6c.$$

Claim 1. First we prove Eq. (7), the lower bound on the whole interval D . This is the set of primes from which p, q are sampled. The following sequence of equations (1) applies Dusart's bounds from above, (2) simplifies logarithms (where e is the base of $\ln$), (3) rearranges terms, and (4) uses simple hardcoded constants to replace some expressions involving e . We have:

$$\begin{aligned}
D &= \pi(2^b) - \pi(2^{b-1}) \\
&\geq \frac{2^b}{\ln(2^b) - 1} - \frac{2^{b-1}}{\ln(2^{b-1}) - 1.1} \\
&= \frac{2^b}{b/\log_2(e) - 1} - \frac{2^{b-1}}{(b-1)/\log_2(e) - 1.1} \\
&= \frac{\log_2(e)2^b}{b - \log_2(e)} - \frac{\log_2(e)2^{b-1}}{(b-1) - 1.1\log_2(e)} \\
&\geq \frac{\log_2(e)2^b}{b - 1.4} - \frac{\log_2(e)2^{b-1}}{b - 2.6} \\
&= \log_2(e)2^{b-1} \left(\frac{2}{b - 1.4} - \frac{1}{b - 2.6} \right) \\
&= \log_2(e)2^{b-1} \left(\frac{2(b - 2.6) - 1(b - 1.4)}{(b - 1.4)(b - 2.6)} \right) \\
&= \log_2(e)2^{b-1} \left(\frac{b - 3.8}{(b - 1.4)(b - 2.6)} \right)
\end{aligned}$$

where this final line is the desired bound.

Claim 2. Next we prove Eq. (8), the upper bound on the number of primes q that are close to p . (This is also an upper bound on such primes q that are in the interval of interest.) We apply a similar sequence of simplifications as in the prior claim. We additionally use the facts that $N^{1/2} < 2^b$ (since each of $p, q < 2^b$), that $2^{b-1} < p < 2^b$, and that $\ln(1 - c)$ for $0 < c \leq 1/4$ is *negative*. We have:

$$\begin{aligned}
Q &= \pi(p + c \cdot N^{1/2}) - \pi(p - c \cdot N^{1/2}) \\
&\leq \pi(p + c2^b) - \pi(p - c2^b) \\
&\leq \frac{p + c2^b}{\ln(p + c2^b) - 1.1} - \frac{p - c2^b}{\ln(p - c2^b) - 1} \\
&\leq \frac{p + c2^b}{\ln(2^{b-1} + c2^b) - 1.1} - \frac{p - c2^b}{\ln(2^b - c2^b) - 1} \\
&= \frac{p + c2^b}{\ln(2^{b-1}(1 + 2c)) - 1.1} - \frac{p - c2^b}{\ln(2^b(1 - c)) - 1}
\end{aligned}$$

$$\begin{aligned}
&= \frac{p + c2^b}{\ln(2^{b-1}) + \ln(1 + 2c) - 1.1} - \frac{p - c2^b}{\ln(2^b) + \ln(1 - c) - 1} \\
&\leq \frac{p + c2^b}{\ln(2^{b-1}) - 1.1} - \frac{p - c2^b}{\ln(2^b) - 1} \\
&= \frac{p + c2^b}{(b-1)/\log_2(e) - 1.1} - \frac{p - c2^b}{b/\log_2(e) - 1} \\
&= \frac{\log_2(e)p + \log_2(e)c2^b}{(b-1) - 1.1\log_2(e)} - \frac{\log_2(e)p - \log_2(e)c2^b}{b - \log_2(e)} \\
&\leq \frac{\log_2(e)p + \log_2(e)c2^b}{b - 2.6} - \frac{\log_2(e)p - \log_2(e)c2^b}{b - 1.4} \\
&= \log_2(e) \left(\frac{p + c2^b}{b - 2.6} - \frac{p - c2^b}{b - 1.4} \right) \\
&= \log_2(e) \left(\frac{(p + c2^b)(b - 1.4) - (p - c2^b)(b - 2.6)}{(b - 2.6)(b - 1.4)} \right) \\
&= \log_2(e) \left(\frac{pb - 1.4p + bc2^b - 1.4c2^b - (pb - 2.6p - bc2^b + 2.6c2^b)}{(b - 2.6)(b - 1.4)} \right) \\
&= \log_2(e) \left(\frac{1.2p + 2bc2^b - 4c2^b}{(b - 2.6)(b - 1.4)} \right) \\
&\leq \log_2(e) \left(\frac{1.2 \cdot 2^b + 2bc2^b - 4c2^b}{(b - 2.6)(b - 1.4)} \right) \\
&\leq \log_2(e) 2^{b-1} \left(\frac{2.4 + 4bc}{(b - 2.6)(b - 1.4)} \right)
\end{aligned}$$

where the last line is the desired bound. Thus equations (7) and (8) both hold, completing the proof of Lemma 9. $\square$